

AppSec Security Review:

`extractable` Flag Misconfiguration



CLASSIFICATION:	SECURITY CRITICAL
SEVERITY:	HIGH (Reverted Immediately)
TARGET FILE:	sgraph_ai_app_send__ui__user/.../vault-crypto.js
REFERENCE FILE:	sgraph_ai_app_send__ui__admin/.../pki-keys.js
VERSION:	v0.5.19
DATE:	22 February 2026
REVIEW TEAM:	Explorer / AppSec

Executive Summary: The Risk and The Resolution

THE EVENT



During a bug fix (commit `a64275e`), the `extractable` parameter for asymmetric key generation was erroneously switched from `false` to `true`.

THE RISK



- Introduced a High Severity vulnerability.
- Allowed for potential permanent compromise of user identity via private key exfiltration.
- Violated privacy claims in the Investor Briefing Pack.

THE RESOLUTION

- Change Reverted Immediately.
- Root cause—a technical misunderstanding of the W3C Spec—corrected.
- New code review gates proposed for `vault-crypto.js`.

The Trigger: Commit `a64275e`

1  deriveVaultCacheKey() ⓘ

... @@ -1,7,-@@ {

2 1 },

1 - extractable: false,

2 + extractable: true,

3 3 algo: 'RSA-OAEP',

4 4 usage: ['encrypt', 'decrypt']

5 5 }

`T`

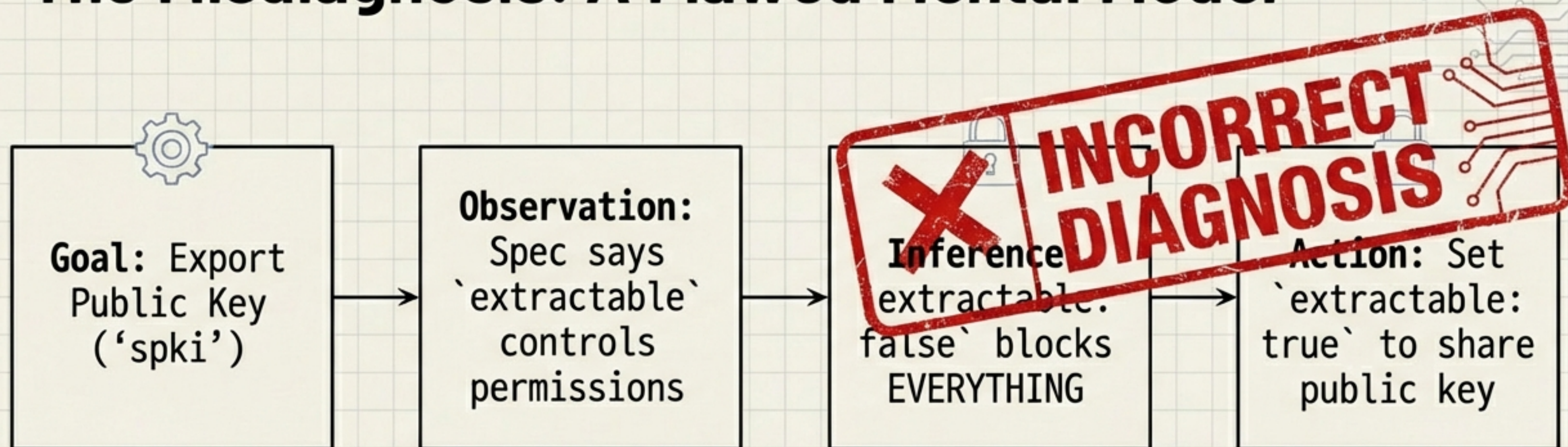
CONTEXT & ASSUMPTION

The developer attempted to fix a bug in `deriveVaultCacheKey()`. They encountered `crypto.subtle.exportKey('spki', publicKey)` and assumed it required the flag to be true.

Assumption: “If extractable is false, the browser throws an InvalidAccessError for ALL key exports.”

Verdict: This assumption was the root cause.

The Misdiagnosis: A Flawed Mental Model



Previous Dev Review (Bug #6): "Web Crypto extractable flag affects public key export — not just private keys." (This was false).

The Reality: W3C Spec for Asymmetric Keys

Asymmetric Key Pair



Public Key

Property [[extractable]]: HARDCODED TO TRUE

Implication:

``crypto.subtle.exportKey('spki', ...)``
ALWAYS works.

Note: Unaffected by user parameters.



Private Key

Property [[extractable]]: USER DEFINED

Implication:

``crypto.subtle.exportKey('pkcs8', ...)``
ONLY works if set to true.

You never need to compromise private key security to share a public key.

Empirical Evidence: The Reference Pattern

vault-crypto.js / Unsafe

```
1  
2 // Commit a64275e (Reverted)  
3 ← extractable: true, // INCORRECT FIX  
4  
5
```

Developer believed this was necessary for export.

pki-keys.js / Admin UI

```
2 // Line 92  
3 → extractable: false,  
4 ...  
5 // Line 93  
exportPublicKeyPEM(keyPair.publicKey);
```

Successfully exports PEM with `extractable: false`.

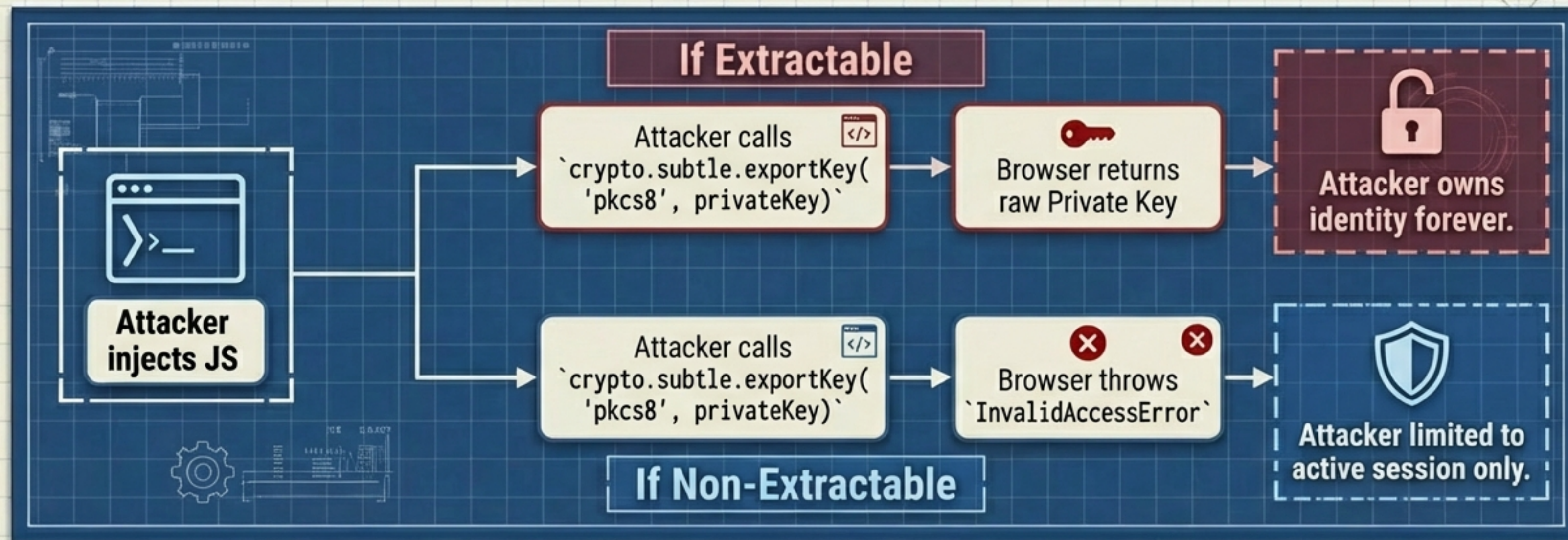
The Admin UI proves that the `InvalidAccessError` fear was unfounded.

Blast Radius: The 'What If?' Scenario

Metric	<code>`extractable: false`</code> (Correct)	<code>`extractable: true`</code> (Risk)
Private Key Export	Blocked by Browser	ALLOWED – Any JS can extract
XSS Impact	Session-bounded (Signing only)	PERMANENT TOTAL KEY COMPROMISE
Recovery Cost	Patch the XSS vulnerability	Rotate keys & Re-encrypt ALL vault data



The XSS Kill Chain



“Extractable keys turn temporary vulnerabilities into permanent disasters.”

Business & Compliance Fallout

Internal Standards

Decision D-PKI-002

VIOLATION

Breaches Milestone:
First PKI Message Exchange

External Trust

Investor Briefing Pack (v0.5.8)

FALSIFIED

Contradicts privacy claims
regarding architecture.

“This is not just a code style issue; it is a breach of trust.”

The Lesson Corrected

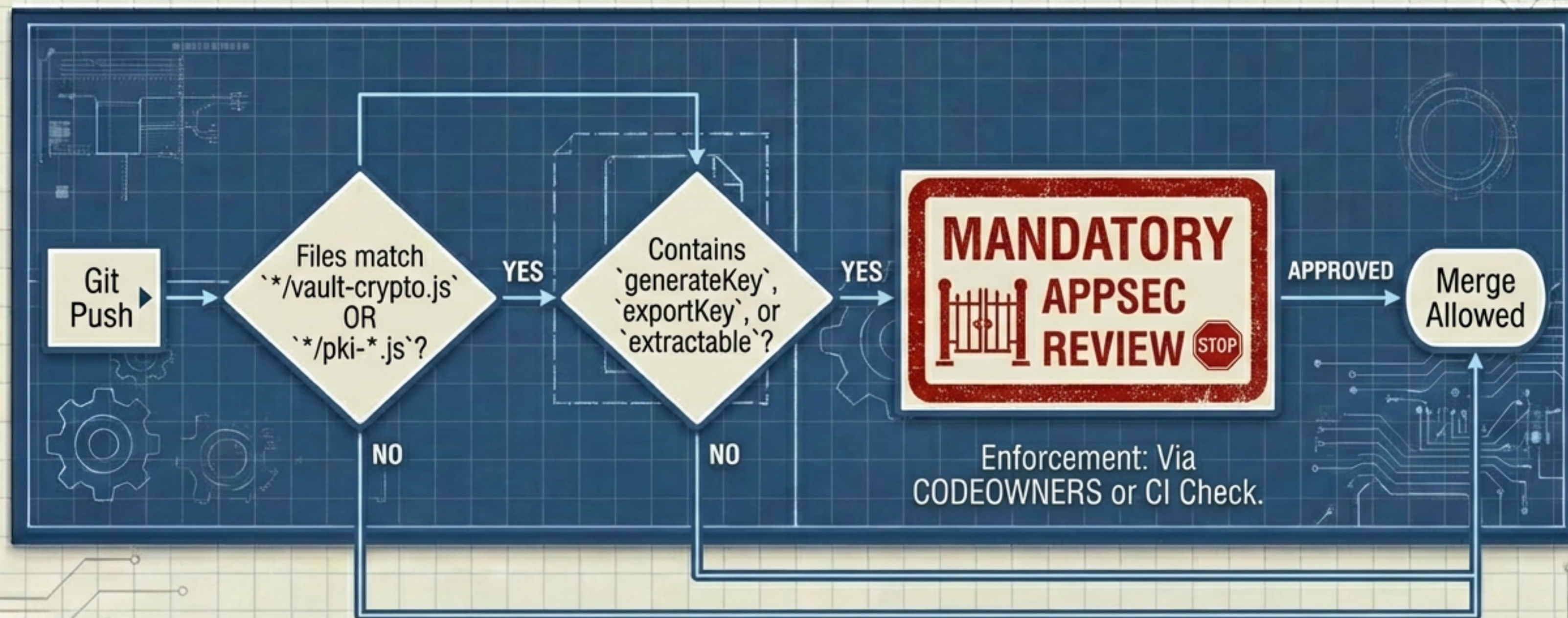
Single Source of Truth

- For asymmetric key pairs, the `extractable` parameter **ONLY** controls private key exportability.
- The **public key** is **ALWAYS extractable** per the W3C spec.

Operational Note:

- `exportKey('spki', publicKey)` works regardless of the flag.
- Previous documentation stating otherwise has been deprecated.

Process Recommendation: New Code Gates



Summary & References

RESOLVED / REVERTED

Decision D-PKI-002: team/roles/historian/...

Architect Plan: team/roles/architect/.../browser-pki-key-management.md

Correct Pattern: .../admin/.../pki-keys.js:92

Investor Pack: library/alchemist/.../v0.5.8__investor-briefing-pack...

Compliance with D-PKI-002 restored.